

1 Sección 1.4: Prestaciones y Rendimiento del Computador

El estudio de las prestaciones en la arquitectura de computadores define los criterios científicos para evaluar, comparar y optimizar el comportamiento de un sistema informático. A diferencia de las nociones comerciales intuitivas, el rendimiento se mide bajo métricas matemáticas rigurosas que vinculan el tiempo de ejecución del software con las restricciones del silicio.

1.1 Definición Formal de Rendimiento y Tiempo de Ejecución

En el ámbito computacional, la métrica primaria de prestaciones es el **Tiempo de Ejecución** o **Tiempo de Respuesta**, definido como el intervalo temporal total que transcurre entre el inicio y la finalización de una tarea (incluyendo accesos a memoria, operaciones de E/S y sobrecarga del sistema operativo).

Matemáticamente, el **Rendimiento (Performance)** de una máquina se define de forma axiomática como el inverso multiplicativo de su Tiempo de Ejecución:

$$\text{Rendimiento}_X = \frac{1}{\text{Tiempo de Ejecucion}_X} \quad (1)$$

Esta relación implica que a menor tiempo de procesamiento, mayor serán las prestaciones del hardware.

1.2 Rendimiento Relativo (Métrica de Speedup)

Para comparar de forma analítica el desempeño de dos computadoras distintas (X e Y) ejecutando una carga de trabajo idéntica, se establece la relación escalar conocida como **Factor de Mejora** o **Speedup**:

$$\text{Speedup} = \frac{\text{Rendimiento}_X}{\text{Rendimiento}_Y} = \frac{\text{Tiempo de Ejecucion}_Y}{\text{Tiempo de Ejecucion}_X} \quad (2)$$

Si el resultado de esta ecuación es mayor a 1,0, se afirma formalmente que el Computador X es “ n veces más rápido” que el Computador Y .

1.3 El Reloj del Procesador y sus Magnitudes Físicas

Los procesadores digitales están gobernados por una señal de reloj constante generada por un oscilador de cuarzo. Los pulsos de esta señal sincronizan las operaciones elementales de las compuertas lógicas en los transistores del procesador. El comportamiento físico del reloj se rige por dos variables mutuamente inversas:

- **Periodo de Ciclo de Reloj (T_c):** Es la duración temporal infinitesimal de un único ciclo completo de la onda de reloj, medida comúnmente en nanosegundos ($1 \text{ ns} = 10^{-9} \text{ s}$) o picosegundos ($1 \text{ ps} = 10^{-12} \text{ s}$).
- **Frecuencia de Reloj (f):** Es la cantidad de ciclos de onda completados por unidad de segundo. Se expresa formalmente en Hertz (Hz), donde un Hertz equivale estrictamente a un ciclo por segundo (ciclos/s). Sus múltiplos industriales son MegaHertz ($1 \text{ MHz} = 10^6 \text{ ciclos/s}$) o GigaHertz ($1 \text{ GHz} = 10^9 \text{ ciclos/s}$).

$$T_c \text{ (s)} = \frac{1}{f \left(\frac{\text{ciclos}}{\text{s}} \right)} \iff f \left(\frac{\text{ciclos}}{\text{s}} \right) = \frac{1}{T_c \text{ (s)}} \quad (3)$$

1.4 La Ecuación Fundamental del Tiempo de CPU

El tiempo total en que la CPU ejecuta activamente código de un programa determinado, excluyendo tiempos de espera por periféricos o hilos ajenos, se denomina **Tiempo de CPU**. Este valor se descompone analíticamente mediante el análisis dimensional de las variables físicas del sistema.

Si conocemos los ciclos totales requeridos por el programa y la frecuencia del reloj en su equivalencia fundamental de unidades (ciclos/s), la ecuación se expresa como:

$$\text{Tiempo de CPU (s)} = \frac{\text{Ciclos de Reloj de la CPU (ciclos)}}{f \left(\frac{\text{ciclos}}{\text{segundo}} \right)} \quad (4)$$

Para profundizar en la interacción entre las capas del computador, se introduce la variable **IC (Instruction Count)** o conteo absoluto de instrucciones del programa, y la variable **CPI (Cycles Per Instruction)**, que representa los ciclos de reloj promedio necesarios para procesar una sola instrucción. Al incorporar la potencia analítica de las unidades de software y hardware, la ecuación final demuestra la consistencia dimensional del sistema:

$$\text{Tiempo de CPU (s)} = \frac{\text{IC (instrucciones)} \times \text{CPI} \left(\frac{\text{ciclos}}{\text{instruccion}} \right)}{f \left(\frac{\text{ciclos}}{\text{segundo}} \right)} \quad (5)$$

Nota de validación de dimensiones: Al multiplicar instrucciones por (ciclos/instrucción) en el numerador se obtienen ciclos netos. Al dividir el resultado entre la frecuencia (ciclos/segundo), la unidad de ciclos se cancela matemáticamente por la regla de la doble C, dejando el resultado final expresado correctamente en **segundos**.

1.5 Matriz de Co-dependencia de los Factores de Rendimiento

La optimización del rendimiento no depende exclusivamente del hardware o del software aislados, sino de la interacción simbiótica de múltiples factores técnicos detallados a continuación:

Componente del Sistema	Afecta IC	Afecta CPI	Afecta f	Justificación Arquitectónica
Algoritmo	Sí	Sí	No	Determina el conteo total de operaciones lógicas primarias y la linealidad del flujo de datos.
Lenguaje de Prog.	Sí	No	No	Los lenguajes de alto nivel abstractos aumentan el IC frente a lenguajes compilados nativos.
Compilador	Sí	Sí	No	Traduce el código fuente a ensamblador; optimiza el orden de instrucciones y uso de registros.
ISA (Set Instrucciones)	Sí	Sí	Sí	Define la complejidad estructural del hardware y el número de ciclos base por familia.
Microarquitectura	No	Sí	Sí	Determina la implementación de la pila, la segmentación (pipelining) y caches.
Semiconductores	No	No	Sí	La tecnología de litografía física (ej. 4nm vs 7nm) limita la frecuencia máxima del silicio.

Cuadro 1: Matriz de impacto de variables de software y hardware sobre el rendimiento.

1.6 CPI Efectivo en Mezclas Estadísticas de Instrucciones

En arquitecturas de computadores reales (multiciclo o segmentadas con penalizaciones por riesgos), las instrucciones no consumen la misma cantidad de ciclos. Las operaciones lógicas en registros son veloces, mientras que los accesos a la memoria principal (*Load/Store*) demandan tiempos significativamente superiores.

Para calcular el comportamiento macroscópico de un programa real, se realiza una media ponderada que calcula el **CPI Efectivo** en función de la frecuencia relativa de aparición (F_i) de cada clase de instrucción i :

$$\text{CPI}_{\text{efectivo}} \left(\frac{\text{ciclos}}{\text{instruccion}} \right) = \sum_{i=1}^n (\text{CPI}_i \times F_i) = \frac{\text{Ciclos Totales de CPU}}{\text{IC}} \quad (6)$$

Donde $F_i = \frac{\text{IC}_i}{\text{IC}}$, cumpliendo estrictamente con la restricción probabilística de normalización: $\sum_{i=1}^n F_i = 1,0$.

1.7 La Falacia de las Métricas MIPS y MFLOPS

Históricamente se han empleado métricas comerciales simplistas para catalogar la velocidad de los procesadores, siendo la más extendida el **MIPS (Millones de Instrucciones Por Segundo)**. Su expresión matemática formal es:

$$\text{MIPS} = \frac{\text{IC}}{\text{Tiempo de Ejecucion} \times 10^6} = \frac{f\left(\frac{\text{ciclos}}{\text{s}}\right)}{\text{CPI} \left(\frac{\text{ciclos}}{\text{instruccion}} \right) \times 10^6} \quad (7)$$

El libro de Patterson & Hennessy demuestra de forma categórica que el MIPS es una métrica **falaz e inválida** para comparar procesadores de diferentes familias debido a tres factores críticos:

1. No toma en cuenta la capacidad de cómputo real de cada instrucción (un procesador CISC podría realizar una tarea compleja con una sola instrucción, mientras un RISC requeriría cinco instrucciones breves, arrojando este último un MIPS falsamente superior).
2. Puede variar de forma inversa al rendimiento real de la computadora en un mismo programa si se modifican las optimizaciones del compilador.
3. Desconoce la naturaleza de la mezcla de instrucciones del código ejecutado.

Para entornos científicos con operaciones de coma flotante, se utiliza la tasa de **MFLOPS (Millones de Operaciones de Punto Flotante Por Segundo)**, la cual restringe la medición exclusivamente a instrucciones aritméticas complejas reales, eludiendo instrucciones de control de flujo o saltos.

1.8 La Ley de Amdahl y la Optimización de Prestaciones

Un principio inmutable en el diseño de computadoras dictamina que las mejoras de hardware implementadas en un componente específico están limitadas por la fracción de tiempo en que dicho componente es verdaderamente utilizado por el software. Este postulado se modela mediante la **Ley de Amdahl**:

$$\text{Tiempo de CPU}_{\text{Nuevo}} = \frac{\text{Tiempo de CPU}_{\text{Afectado}}}{\text{Factor de Mejora}} + \text{Tiempo de CPU}_{\text{No Afectado}} \quad (8)$$

Definiendo la fracción de tiempo original que se beneficia de la mejora como f_{mejora} y el incremento de velocidad local del módulo optimizado como k , la ecuación del Speedup global resultante se expresa como:

$$\text{Speedup}_{\text{Global}} = \frac{1}{(1 - f_{\text{mejora}}) + \frac{f_{\text{mejora}}}{k}} \quad (9)$$

Este modelo formaliza la regla de diseño arquitectónico fundamental: “*Optimiza el caso común y mantén el diseño balanceado*”.

1.9 Ejemplos de Cálculo Analítico (Ejercicios Resueltos Tipo Examen)

1.9.1 Ejemplo 1: Evaluación Compleja de Dos Arquitecturas ISA

Enunciado: Una estación de trabajo de la Facultad de Ingeniería procesa un programa de análisis estructural. Se evalúan dos propuestas de diseño de hardware bajo el mismo código fuente:

- **Diseño Alfa:** Posee una frecuencia de reloj de 4,0 GHz ($4,0 \times 10^9$ ciclos/s) y su arquitectura requiere un CPI promedio de 1,8 para este programa.
- **Diseño Beta:** Implementa un compilador optimizado que logra reducir el conteo de instrucciones totales a un 80 % en comparación con Alfa ($IC_{Beta} = 0,8 \times IC_{Alfa}$). Sin embargo, debido a los cambios estructurales, la frecuencia de reloj debió reducirse a 3,2 GHz ($3,2 \times 10^9$ ciclos/s) y su CPI se incrementó a 2,2.

Determine analíticamente cuál diseño es más rápido y calcule el Speedup global exacto.

Resolución: Planteamos la ecuación fundamental del tiempo de ejecución de la CPU para ambos sistemas en función del conteo de instrucciones base IC_{Alfa} y las unidades fundamentales:

$$Tiempo_{Alfa} = \frac{IC_{Alfa} \times 1,8 \text{ ciclos}}{4,0 \times 10^9 \text{ ciclos/s}} = IC_{Alfa} \times (4,5 \times 10^{-10}) \text{ segundos} \quad (10)$$

Para el Diseño Beta, incorporamos las variaciones proporcionales de las variables:

$$Tiempo_{Beta} = \frac{(0,8 \times IC_{Alfa}) \times 2,2 \text{ ciclos}}{3,2 \times 10^9 \text{ de ciclos/s}} \quad (11)$$

$$Tiempo_{Beta} = \frac{IC_{Alfa} \times 1,76 \text{ ciclos}}{3,2 \times 10^9 \text{ ciclos/s}} = IC_{Alfa} \times (5,5 \times 10^{-10}) \text{ segundos} \quad (12)$$

Procedemos a evaluar la métrica de Speedup relativo dividiendo el tiempo de ejecución del sistema más lento entre el más rápido:

$$Speedup = \frac{Tiempo_{Beta}}{Tiempo_{Alfa}} = \frac{IC_{Alfa} \times 5,5 \times 10^{-10} \text{ s}}{IC_{Alfa} \times 4,5 \times 10^{-10} \text{ s}} = \frac{5,5}{4,5} \approx 1,222 \quad (13)$$

Conclusión Analítica: El Diseño Alfa es más rápido que el Diseño Beta por un factor exacto de 1,22 (**un 22,2 % superior**). A pesar de que el Diseño Beta ejecuta un 20 % menos instrucciones, la degradación simultánea en su frecuencia de reloj y el incremento en los ciclos por instrucción neutralizaron la optimización del compilador.

1.9.2 Ejemplo 2: Mezcla de Instrucciones y Optimización de Compilación

Enunciado: Un procesador MIPS implementa tres clases funcionales de instrucciones con sus respectivos CPI de diseño: Clase A (CPI = 1), Clase B (CPI = 2) y Clase C (CPI = 3). Dos compiladores experimentales generan código máquina para un mismo algoritmo con las siguientes distribuciones absolutas:

- **Compilador 1:** Genera 5×10^6 inst. de Clase A, 2×10^6 de Clase B y 3×10^6 de Clase C.
- **Compilador 2:** Genera 10×10^6 inst. de Clase A, 1×10^6 de Clase B y 1×10^6 de Clase C.

Calcule el CPI efectivo para cada caso y dictamine cuál compilador produce un rendimiento óptimo de la CPU.

Resolución Compilador 1:

$$IC_{Total1} = (5 + 2 + 3) \times 10^6 = 10 \times 10^6 \text{ instrucciones} \quad (14)$$

$$Ciclos_{Comp1} = \sum (IC_i \times CPI_i)$$

$$Ciclos_{Comp1} = (5 \times 10^6 \times 1) + (2 \times 10^6 \times 2) + (3 \times 10^6 \times 3)$$

$$Ciclos_{Comp1} = (5 + 4 + 9) \times 10^6 = 18 \times 10^6 \text{ ciclos de reloj}$$

$$CPI_{Efectivo1} = \frac{Ciclos_{Comp1}}{IC_{Total1}} = \frac{18 \times 10^6 \text{ ciclos}}{10 \times 10^6 \text{ inst.}} = 1,80 \left(\frac{\text{ciclos}}{\text{instruccion}} \right) \quad (15)$$

Resolución Compilador 2:

$$IC_{Total2} = (10 + 1 + 1) \times 10^6 = 12 \times 10^6 \text{ instrucciones} \quad (16)$$

$$Ciclos_{Comp2} = (10 \times 10^6 \times 1) + (1 \times 10^6 \times 2) + (1 \times 10^6 \times 3)$$

$$Ciclos_{Comp2} = (10 + 2 + 3) \times 10^6 = 15 \times 10^6 \text{ ciclos de reloj}$$

$$CPI_{Efectivo2} = \frac{Ciclos_{Comp2}}{IC_{Total2}} = \frac{15 \times 10^6 \text{ ciclos}}{12 \times 10^6 \text{ inst.}} = 1,25 \left(\frac{\text{ciclos}}{\text{instruccion}} \right) \quad (17)$$

Conclusión Analítica: Evaluando los ciclos absolutos de reloj, el **Compilador 2 es superior** debido a que requiere menos ciclos totales de reloj (15×10^6 vs 18×10^6) para ejecutar la misma tarea lógica, traducándose en un menor tiempo de procesamiento efectivo. Este ejercicio ilustra una paradoja clásica de la arquitectura: el Compilador 2 incrementó el conteo de instrucciones totales (IC aumentó de 10 a 12 millones), pero redujo drásticamente su CPI efectivo, logrando una ejecución global netamente más veloz.

1.9.3 Ejemplo 3: Aplicación Teórica de la Ley de Amdahl

Enunciado: El equipo de diseño de un procesador industrial busca optimizar el pipeline matemático. Se analiza implementar un coprocesador vectorial de punto flotante por hardware que ejecuta cálculos aritméticos 10 veces más rápido que la lógica actual ($k = 10$). Mediante análisis de trazas de perfiles del software, se determina que las instrucciones de punto flotante representan el 40 % del tiempo total de CPU del programa de control ($f_{\text{mejora}} = 0,40$). Calcule el Speedup global real del sistema tras la integración física del componente.

Resolución: Aplicamos de forma directa la ecuación escalar de la Ley de Amdahl:

$$\text{Speedup}_{\text{Global}} = \frac{1}{(1 - f_{\text{mejora}}) + \frac{f_{\text{mejora}}}{k}} \quad (18)$$

Sustituyendo los parámetros operacionales provistos por el enunciado técnico ($f_{\text{mejora}} = 0,40$ y $k = 10$):

$$\text{Speedup}_{\text{Global}} = \frac{1}{(1 - 0,40) + \frac{0,40}{10}} = \frac{1}{0,60 + 0,04} \quad (19)$$

$$\text{Speedup}_{\text{Global}} = \frac{1}{0,64} \approx 1,5625 \quad (20)$$

Conclusión Analítica: La integración del nuevo hardware vectorial proporciona un factor de incremento de velocidad global exacto de 1,56 (**un 56,25 % de ganancia en las prestaciones generales del procesador**). Este ejercicio demuestra cuantitativamente los límites impuestos por la Ley de Amdahl: a pesar de haber acelerado un componente local de manera masiva (un 1000 % de rendimiento en coma flotante), el impacto global se vio estrictamente acotado por el 60 % del programa que continuó ejecutándose de forma convencional secuencial.